Question # 01

Imagine a group of players competing in a number guessing game where everyone is confidently convinced that their next guess will magically be correct. The program secretly generates a random number between 1 and 100, and the players take turns guessing the number. If a player guesses incorrectly, the program politely tells them whether their guess was too high or too low, and the turn is passed to the next player so that someone else gets a chance to be wrong. The game continues until a player finally guesses the correct number and wins. Implement this game using **Direct Recursion**, where the recursive function manages the turns and continues the game until the correct guess is made.

Question # 02

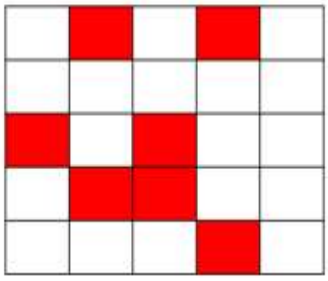
Implement Question # 01 using mutual recursion.

Question # 03

Imagine you have been hired for a "very important" scientific application that has somehow managed to create a multi-dimensional array so complicated that even looking at it makes you question your career choices. Your task is to calculate the sum of all elements in this nested structure using recursion. The array is represented as a jagged array, where each dimension may contain nested arrays of different sizes. Write a C++ function int recursiveArraySum(int* arr[], int sizes[], int dim) that recursively calculates and returns the total sum of all elements. The sizes array contains the size of each dimension, while dim indicates the current dimension being processed. Remember, you are not allowed to simply flatten the array and pretend the problem never existed; recursion must do the hard work.

Question # 04

Imagine you are given a grid representing a maze, where "0"s represent obstacles(red highlighted) or walls, and "1"s represent open paths or empty cells. You have two arrays at your disposal: an empty Solution array and a maze represented by the following grid:



Your task is to guide a lion through a maze from the starting point at (0, 0) to reach a piece of meat located at the destination point (4, 4). The lion is not allowed to move diagonally and can only move through open paths (represented by '1's) while avoiding obstacles (represented by '0's). Please provide a C++ code that uses a backtracking algorithm to navigate the maze and move the lion to the meat. Additionally, display the contents of the Solution array after the lion has reached the meat.

Question # 05

Imagine you are helping a person on a very serious mission: collecting as many flags as possible from a 16×16 grid. Unfortunately, the flags come with unnecessary attitude and refuse to be placed too close to one another. Your task is to design a backtracking algorithm in C++ to find the maximum number of flags that can be placed while following these rules:

- Only one flag can be placed in each row or column.
- Only one flag can be placed in each column.
- No two flags can be placed on the same diagonal, meaning no two flags should be able to threaten each other diagonally.
- The algorithm must use backtracking to explore possible placements and find the maximum number of flags that can be placed.

Implement the algorithm in C++, provide the complete code, and report the maximum number of flags that can be placed on the 16×16 grid. Because apparently, even placing flags requires a full-blown algorithm now.

Question # 06

A drone delivery service is tasked with transporting critical medical supplies from a distribution center (marked 'D') located at the top left corner of a 6x6 grid to a hospital (marked 'H') at the bottom right corner of the grid. The grid represents a terrain that contains various types of areas:

- 'S' indicates smooth terrain, which allows the drone to fly without any issues.
- D represents an area with high wind disturbances. The drone can travel through these cells, but doing so consumes additional battery.
- 'F' indicates areas with no-fly zones due to severe weather (completely blocked for the drone).
- 'H' marks the hospital, the delivery destination.

D	S	S	F	D	F
S	S	S	F	S	D
S	D	S	S	S	F
F	S	F	S	S	F
S	S	S	D	S	F
S	F	S	S	S	H

Rules:

- The drone starts at the top-left D cell and must reach the H cell.
- The drone can move only up, down, left, or right. Diagonal movement is not allowed.
- The drone cannot enter any F cell.
- The drone may travel through D cells, but these cells cause additional battery consumption.
- The drone should not visit the same cell more than once during a single path.
- The objective is to find a valid path from D to H that minimizes the number of high-wind (D) cells visited.
- If multiple paths have the same minimum number of high-wind cells, any one of those paths is acceptable.

Using a recursive/backtracking approach, find the safest possible path from the Distribution Center (D) to the Hospital (H).

Your solution should identify:

- The complete path followed by the drone.
- The number of high-wind (D) cells encountered.
- The cells that are blocked because of no-fly zones (F).
- The final path with the minimum possible number of high-wind cells.

The goal is to find a path for the drone from 'D' to 'H' while avoiding the no-fly zones ('F') and minimizing disruptions from the high wind areas ('D'). The drone should follow the safest route possible, maximizing its battery life by avoiding disturbances.

Question # 07

A curious squirrel begins its journey at position (0,0) in a 5x5 forest, searching for 7 acorns hidden across the terrain. The forest is also scattered with dangerous traps that the squirrel must avoid to survive. Meanwhile, a cunning fox starts at position (2,4) with a goal to collect 6 valuable gems without stumbling into any traps. Both the squirrel and the fox must carefully explore the forest, moving from one spot to another. If they find acorns or gems, they collect them; if they hit a trap, they backtrack and try another direction. The grid represents a terrain that contains various types of areas:

- 'S': Starting point of the squirrel.
- 'F': Starting point of the fox (now at position (2,4)).
- 'A': Acorns (squirrel's goal).
- 'G': Gems (fox's goal).
- 'T': Traps (both must avoid).

S	A	A	T	A
A	T	A	A	G
T	A	T	A	F
A	A	A	T	G
A	T	G	T	G

Using recursion and backtracking, the squirrel and the fox navigate the forest, avoiding traps and collecting their respective items. You need to provide the path for both animals and display two output arrays based on their movements.